Lý thuyết đồ thị cơ bản

Cấu trúc dữ liệu và Giải thuật

Hà Minh Đức

Nội dung

1. Giới thiệu
2. Khái niệm cơ bản
3. Đường đi
4. Thuật toán duyệt
5. Cây

Đồ thị là gì?

Khái niệm

Đồ thị $G = (V, E)$ là một cấu trúc rời rạc gồm tập các đỉnh V và tập các cạnh E nối các cặp đỉnh trong V.

Đồ thị là gì?

Khái niệm

Đồ thị $G = (V, E)$ là một cấu trúc rời rạc gồm tập các đỉnh V và tập các cạnh E nối các cặp đỉnh trong V.

Thành phần và Thuật ngữ

- **Đỉnh (Vertex/Node):** Đối tượng cần biểu diễn.
- **Cạnh (Edge/Arc):** Quan hệ giữa hai đối tượng.
- **Bậc (Degree):** $deg(v)$ là số cạnh kề với đỉnh v.
- **Định lý bắt tay:** $\sum_{v \in V} deg(v) = 2|E|$.

Đồ thị là gì?

Khái niệm

Đồ thị $G = (V, E)$ là một cấu trúc rời rạc gồm tập các đỉnh V và tập các cạnh E nối các cặp đỉnh trong V.

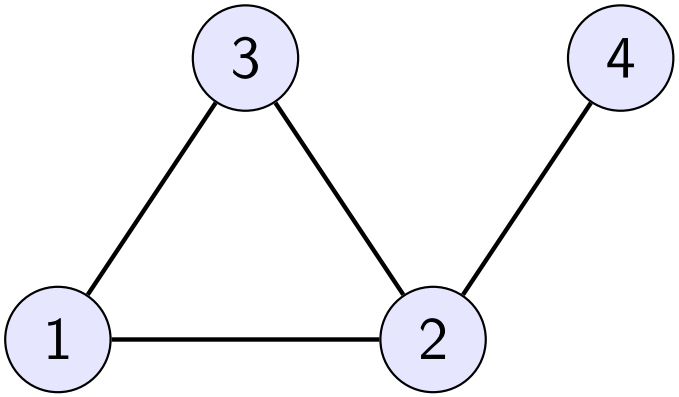
Thành phần và Thuật ngữ

- **Đỉnh (Vertex/Node):** Đối tượng cần biểu diễn.
- **Cạnh (Edge/Arc):** Quan hệ giữa hai đối tượng.
- **Bậc (Degree):** $deg(v)$ là số cạnh kề với đỉnh v.
- **Định lý bắt tay:** $\sum_{v \in V} deg(v) = 2|E|$.

Ví dụ thực tế theo VNOI:

- **Mạng xã hội:** Đỉnh là người dùng, cạnh là quan hệ bạn bè.
- **Giao thông:** Đỉnh là ngã tư, cạnh là các cung đường nối.
- **Lập trình:** Biểu diễn phụ thuộc giữa các module/thư viện.

Ví dụ đồ thị



- **Tập đỉnh:** $V = \{1, 2, 3, 4\}$
- **Tập cạnh:**
  $E = \{(1, 2), (1, 3), (2, 3), (2, 4)\}$
- **Bậc của các đỉnh:**
  - $deg(1) = 2$
  - $deg(2) = 3$ (Đỉnh có bậc cao nhất)
  - $deg(3) = 2, deg(4) = 1$
- **Tổng bậc:**
  $2 + 3 + 2 + 1 = 8 = 2 \times |E|$

Định nghĩa đồ thị

Định nghĩa hình thức

Đồ thị được ký hiệu $G = (V, E)$. Trong đó $V \neq \emptyset$ và E là tập các cặp (u, v) với $u, v \in V$.

Định nghĩa đồ thị

Định nghĩa hình thức

Đồ thị được ký hiệu $G = (V, E)$. Trong đó $V \neq \emptyset$ và E là tập các cặp (u, v) với $u, v \in V$.

- V : Tập các đỉnh (Vertices).
- E : Tập các cạnh (Edges).
- **Đồ thị đơn:** Không có đa cạnh và không có khuyên (loop).
- **Đồ thị có trọng số:** Mỗi cạnh đi kèm một giá trị số $w(u, v)$.

Định nghĩa đồ thị

Định nghĩa hình thức

Đồ thị được ký hiệu $G = (V, E)$. Trong đó $V \neq \emptyset$ và E là tập các cặp (u, v) với $u, v \in V$.

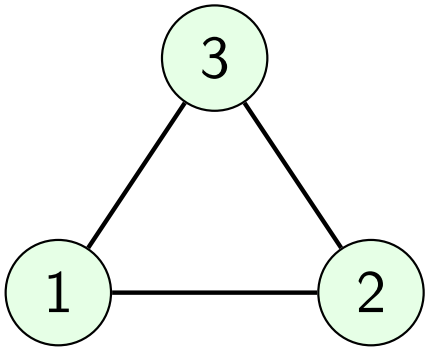
- V : Tập các đỉnh (Vertices).
- E : Tập các cạnh (Edges).
- **Đồ thị đơn:** Không có đa cạnh và không có khuyên (loop).
- **Đồ thị có trọng số:** Mỗi cạnh đi kèm một giá trị số $w(u, v)$.

Cách lưu trữ (Biểu diễn đồ thị)

- **Ma trận kề:** Mảng 2 chiều $A[V][V]$. Tốn $O(V^2)$ bộ nhớ.
- **Danh sách kề:** Mảng các vector/list. Tốn $O(V + E)$ bộ nhớ (*Ưu tiên dùng*).

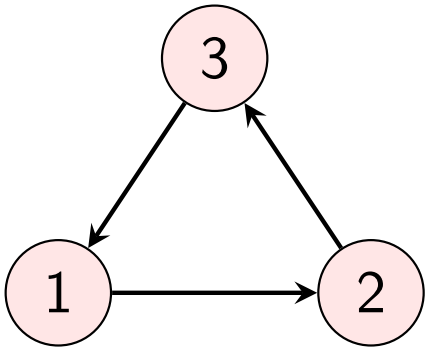
Đồ thị vô hướng

- Cạnh $\{u, v\}$ không có tính thứ tự.
- Quan hệ đối xứng: Nếu có đường từ $u \to v$ thì chắc chắn có $v \to u$.
- **Tính liên thông:** Đồ thị gọi là liên thông nếu có đường đi giữa mọi cặp đỉnh.



Đồ thị có hướng

- Cạnh (u, v) có hướng ($Cung$).
- u là đỉnh đầu, v là đỉnh cuối.
- **Liên thông mạnh:** Mọi cặp (u, v) đều có đường đi từ $u \to v$ và $v \to u$.
- **Bậc:** Gồm bậc vào deg^- và bậc ra deg^+.



Đường đi (Path)

Định nghĩa chi tiết

Đường đi độ dài k từ u đến v là dãy các đỉnh $x_0, x_1, ..., x_k$ sao cho $x_0 = u, x_k = v$ và $(x_i, x_{i+1}) \in E$.

Đường đi (Path)

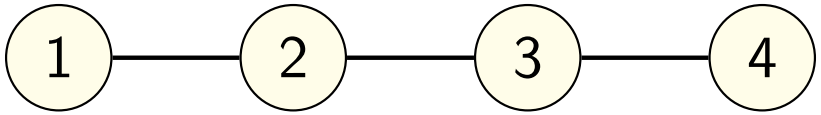
Định nghĩa chi tiết

Đường đi độ dài k từ u đến v là dãy các đỉnh $x_0, x_1, ..., x_k$ sao cho $x_0 = u, x_k = v$ và $(x_i, x_{i+1}) \in E$.

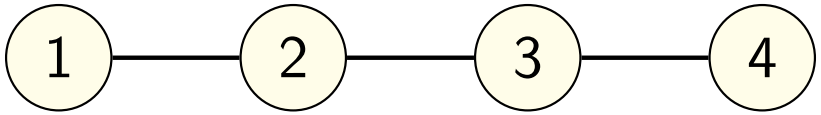
Phân loại

- **Đường đi đơn:** Không có đỉnh nào lặp lại.
- **Đường đi sơ cấp:** Không có cạnh nào lặp lại.
- **Độ dài đường đi:** Số lượng cạnh trên đường đi đó (hoặc tổng trọng số cạnh).

Ví dụ đường đi



Ví dụ đường đi



Đường đi từ 1 tới 4: $1 \to 2 \to 3 \to 4$.

- Số đỉnh: 4
- Số cạnh (Độ dài): 3
- Đây là một **đường đi đơn**.

Chu trình (Cycle)

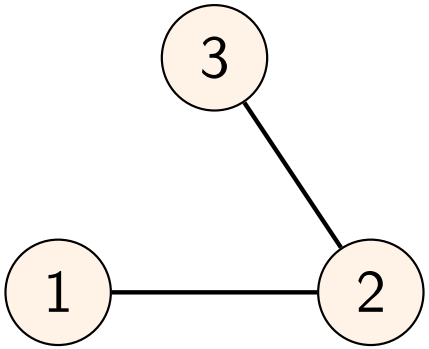
Định nghĩa

Chu trình là đường đi có đỉnh bắt đầu trùng với đỉnh kết thúc ($v_1 \equiv v_k$) và có ít nhất một cạnh.

Chu trình (Cycle)

Định nghĩa

Chu trình là đường đi có đỉnh bắt đầu trùng với đỉnh kết thúc ($v_1 \equiv v_k$) và có ít nhất một cạnh.



Lưu ý: Đồ thị không có chu trình được gọi là *Acyclic graph*. Nếu là đồ thị có hướng không chu trình thì gọi là **DAG** (Directed Acyclic Graph).

Duyệt đồ thị (Graph Traversal)

Mục tiêu: Ghé thăm tất cả các đỉnh có thể đến được từ một đỉnh xuất phát.

- **DFS (Depth First Search):** Ưu tiên đi sâu.
- **BFS (Breadth First Search):** Ưu tiên đi rộng (theo lớp).

Duyệt đồ thị (Graph Traversal)

Mục tiêu: Ghé thăm tất cả các đỉnh có thể đến được từ một đỉnh xuất phát.

- **DFS (Depth First Search):** Ưu tiên đi sâu.
- **BFS (Breadth First Search):** Ưu tiên đi rộng (theo lớp).

Độ phức tạp thuật toán

Sử dụng danh sách kề: $O(V + E)$.
Sử dụng ma trận kề: $O(V^2)$.

Duyệt đồ thị (Graph Traversal)

Mục tiêu: Ghé thăm tất cả các đỉnh có thể đến được từ một đỉnh xuất phát.

- **DFS (Depth First Search):** Ưu tiên đi sâu.
- **BFS (Breadth First Search):** Ưu tiên đi rộng (theo lớp).

Độ phức tạp thuật toán

Sử dụng danh sách kề: $O(V + E)$.
Sử dụng ma trận kề: $O(V^2)$.

Ứng dụng chính:

- Tìm thành phần liên thông.
- Kiểm tra đồ thị hai phía (Bipartite Graph).
- Tìm đường đi ngắn nhất trên đồ thị không trọng số (BFS).

DFS - Duyệt theo chiều sâu

Ý tưởng và Cấu trúc

- Sử dụng **Ngăn xếp (Stack)** hoặc **Đệ quy**.
- Thăm một đỉnh kề, sau đó tiếp tục thăm ngay đỉnh kề của đỉnh đó cho đến khi gặp ngõ cụt mới quay lui (backtrack).

DFS - Duyệt theo chiều sâu

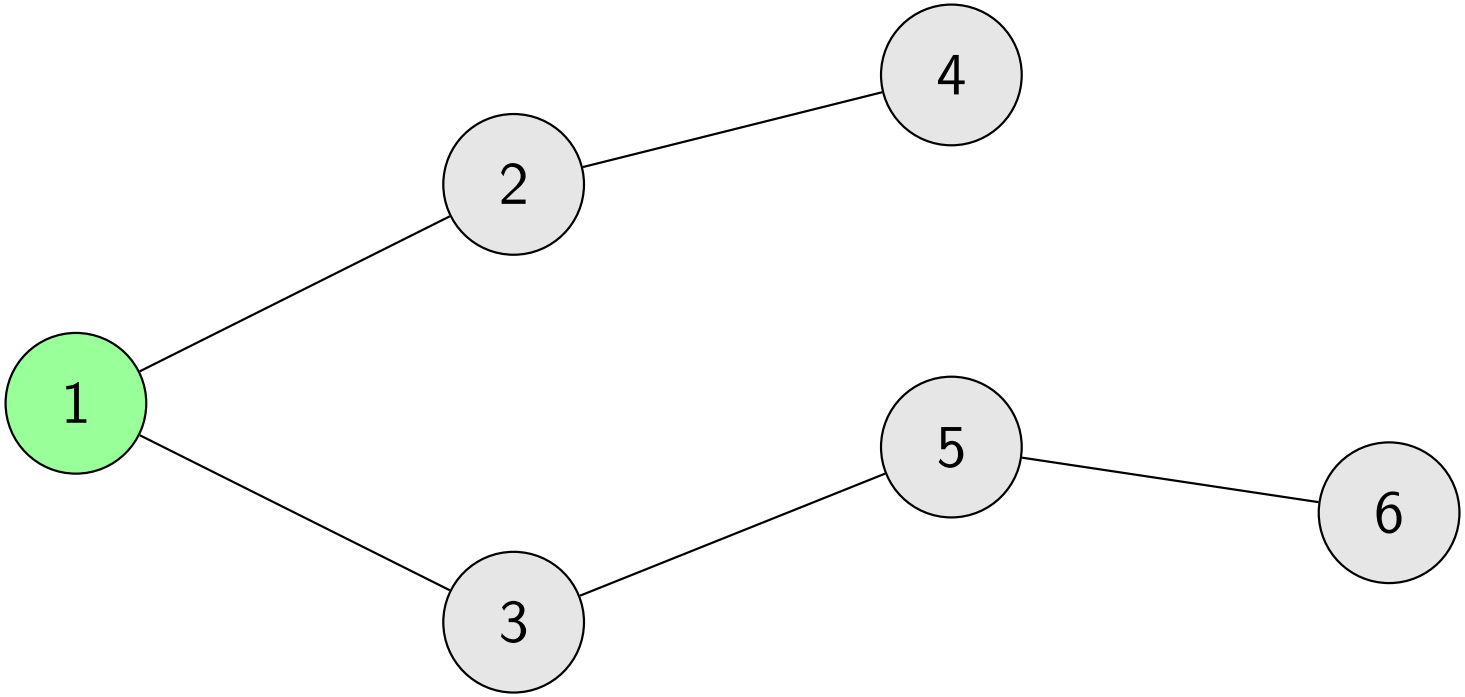
Ý tưởng và Cấu trúc

- Sử dụng **Ngăn xếp (Stack)** hoặc **Đệ quy**.
- Thăm một đỉnh kề, sau đó tiếp tục thăm ngay đỉnh kề của đỉnh đó cho đến khi gặp ngõ cụt mới quay lui (backtrack).

Ứng dụng chuyên sâu:

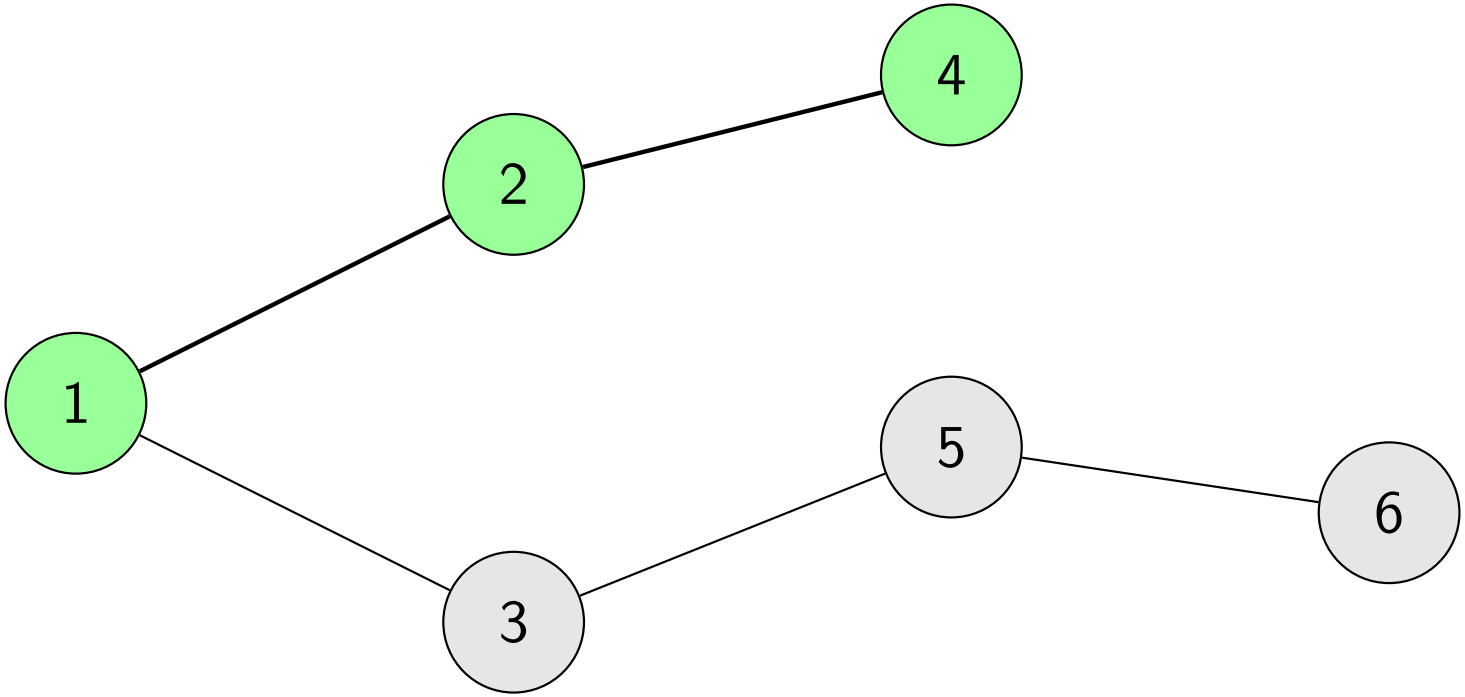
- Tìm các khớp (Articulation points) và cầu (Bridges).
- Phân tách các thành phần liên thông mạnh (Tarjan, Kosaraju).
- Sắp xếp Topo (Topological Sort).

DFS Step 1



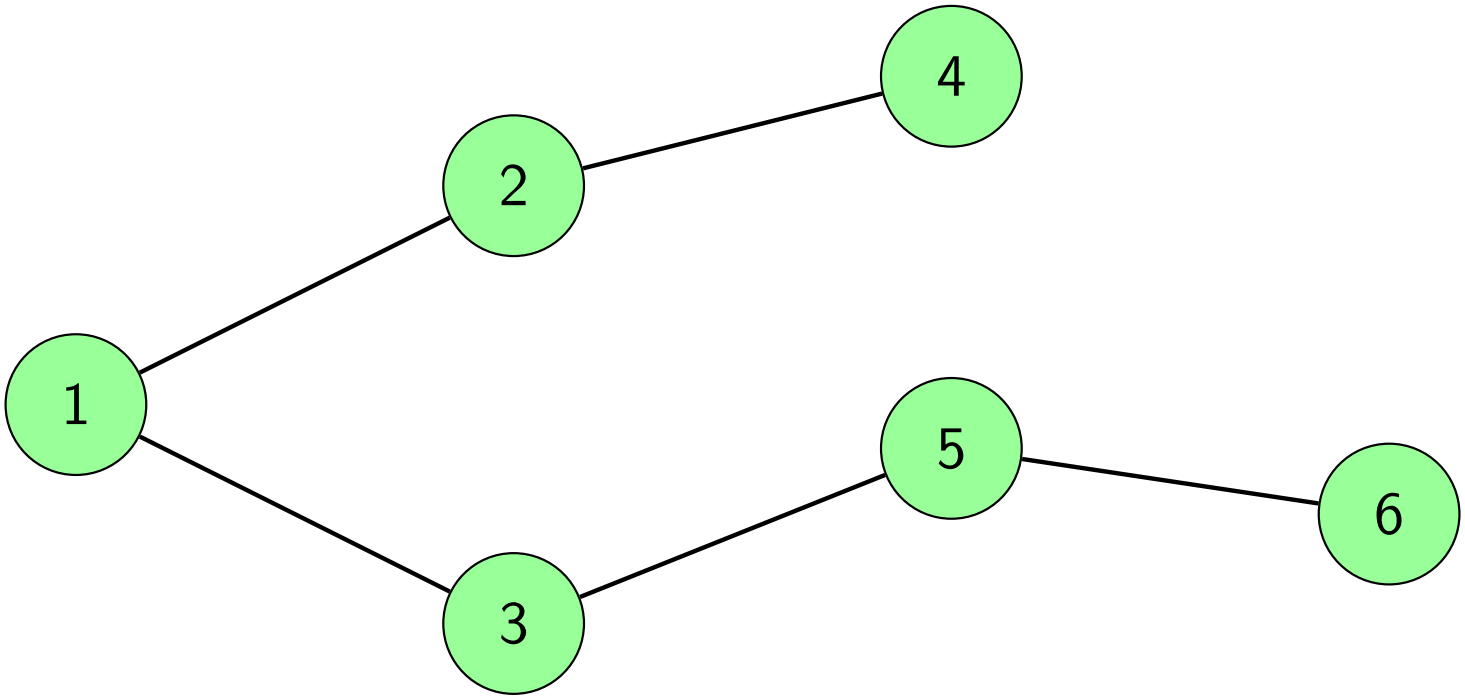
Bắt đầu DFS từ đỉnh **1**

DFS Step 2



DFS đi sâu: **1 → 2 → 4**

DFS Step 3



Backtrack và tiếp tục: **1 → 2 → 4 → quay lui → 3 → 5 → 6**

Mã giả DFS

Cài đặt DFS bằng đệ quy (C++)

```
vector<int> adj[N];
bool visited[N];

void dfs(int u){
    visited[u] = true;
    cout << u << " ";

    for(int v : adj[u]){
        if(!visited[v]){
            dfs(v);
        }
    }
}
```

BFS - Duyệt theo chiều rộng

Ý tưởng và Cấu trúc

- Sử dụng **Hàng đợi (Queue)**.
- Thăm tất cả các đỉnh kề trực tiếp (khoảng cách 1), sau đó mới thăm các đỉnh ở khoảng cách 2, 3...

BFS - Duyệt theo chiều rộng

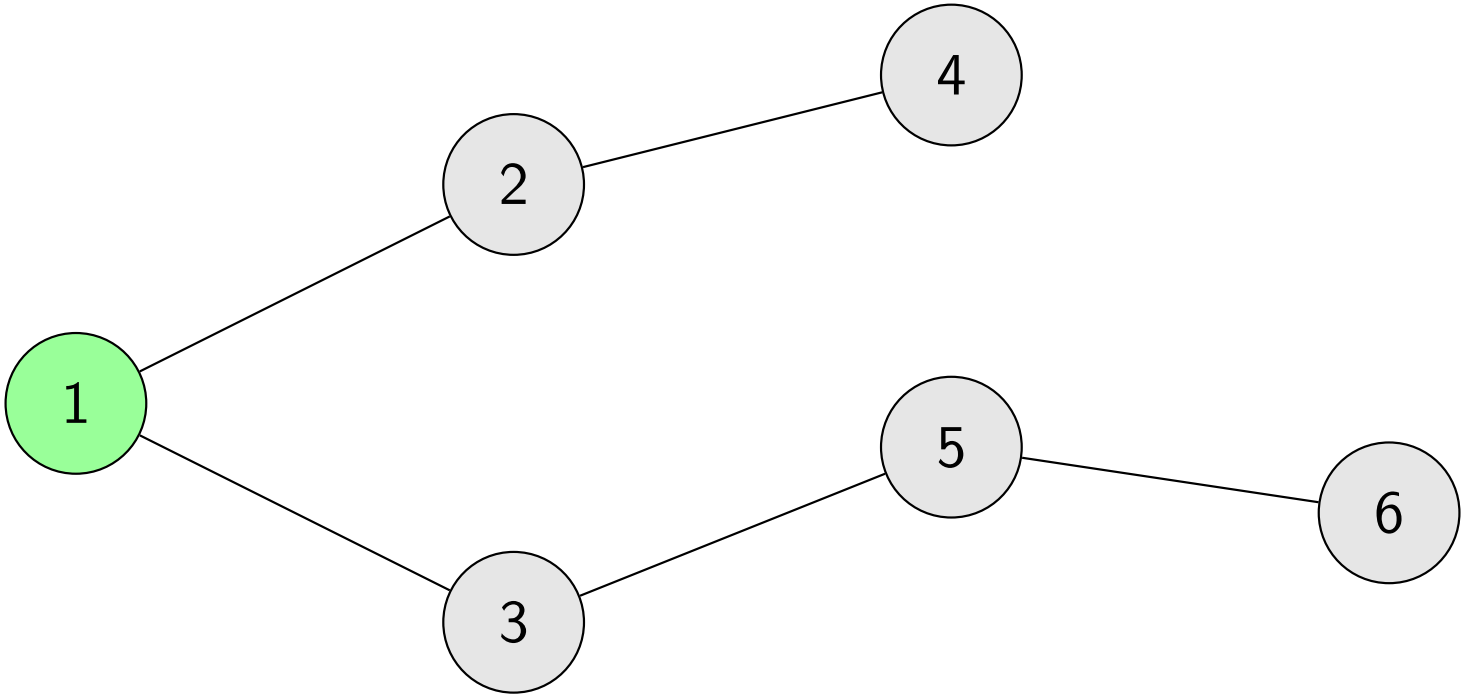
Ý tưởng và Cấu trúc

- Sử dụng **Hàng đợi (Queue)**.
- Thăm tất cả các đỉnh kề trực tiếp (khoảng cách 1), sau đó mới thăm các đỉnh ở khoảng cách 2, 3...

Tính chất quan trọng:

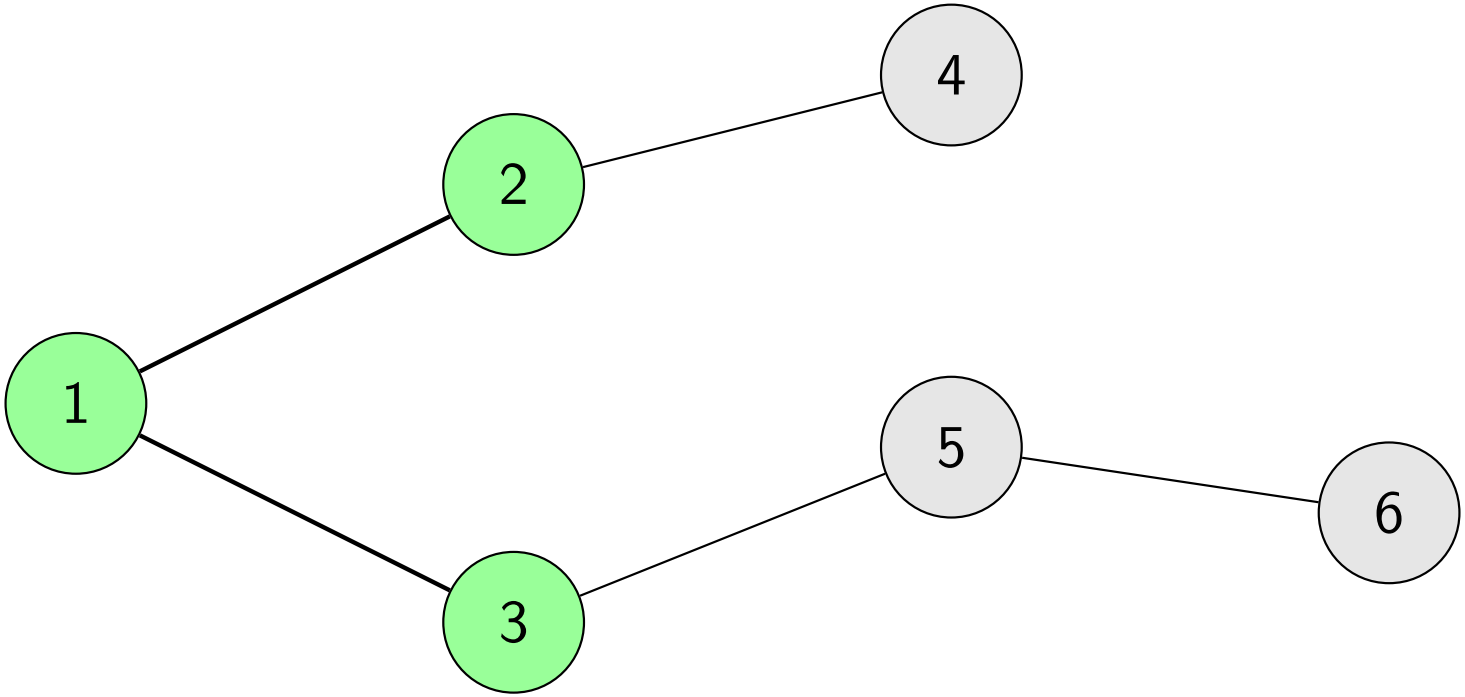
- **Đường đi ngắn nhất:** BFS luôn tìm được đường đi ngắn nhất (tính bằng số cạnh) từ đỉnh nguồn tới mọi đỉnh khác.
- **Cây BFS:** Tập hợp các cạnh được sử dụng để thăm đỉnh mới tạo thành một cây.

BFS Step 1



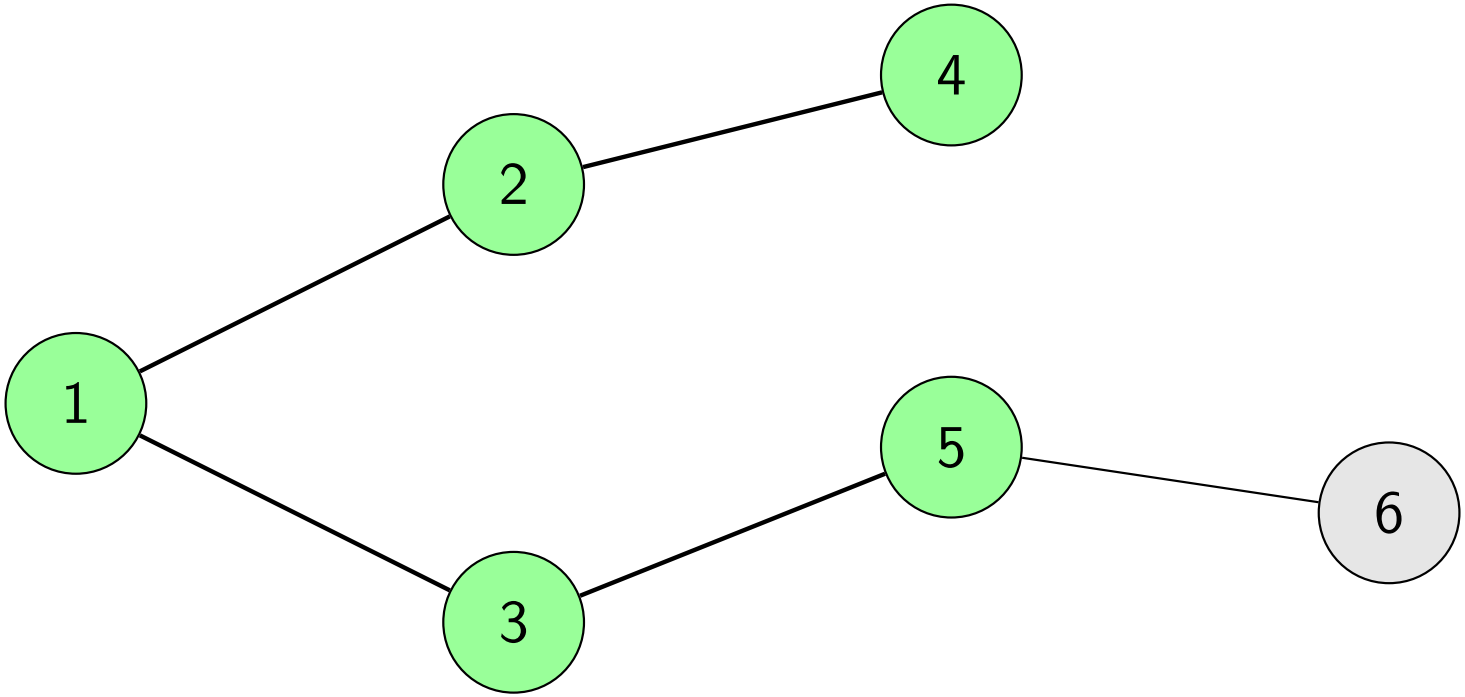
Bắt đầu BFS từ đỉnh **1**
Queue: [1]

BFS Step 2



Thăm các đỉnh kề của 1: **2, 3**
Queue: [2,3]

BFS Step 3



Từ **2 và 3** thăm tiếp: **4, 5**
Queue: [4,5]

Mã mẫu BFS

Cài đặt BFS bằng hàng đợi (C++)

```
vector<int> adj[N];bool visited[N];
void bfs(int s){
    queue<int> q;visited[s] = true;
    q.push(s);
    while(!q.empty()){
        int u = q.front();
        q.pop();
        cout << u << " ";
        for(int v : adj[u])if(!visited[v]){
            visited[v] = true;
            q.push(v);
        }
    }
}
```

Cây (Tree)

Định nghĩa

Cây là một đồ thị vô hướng, liên thông và **không có chu trình**.

Cây (Tree)

Định nghĩa

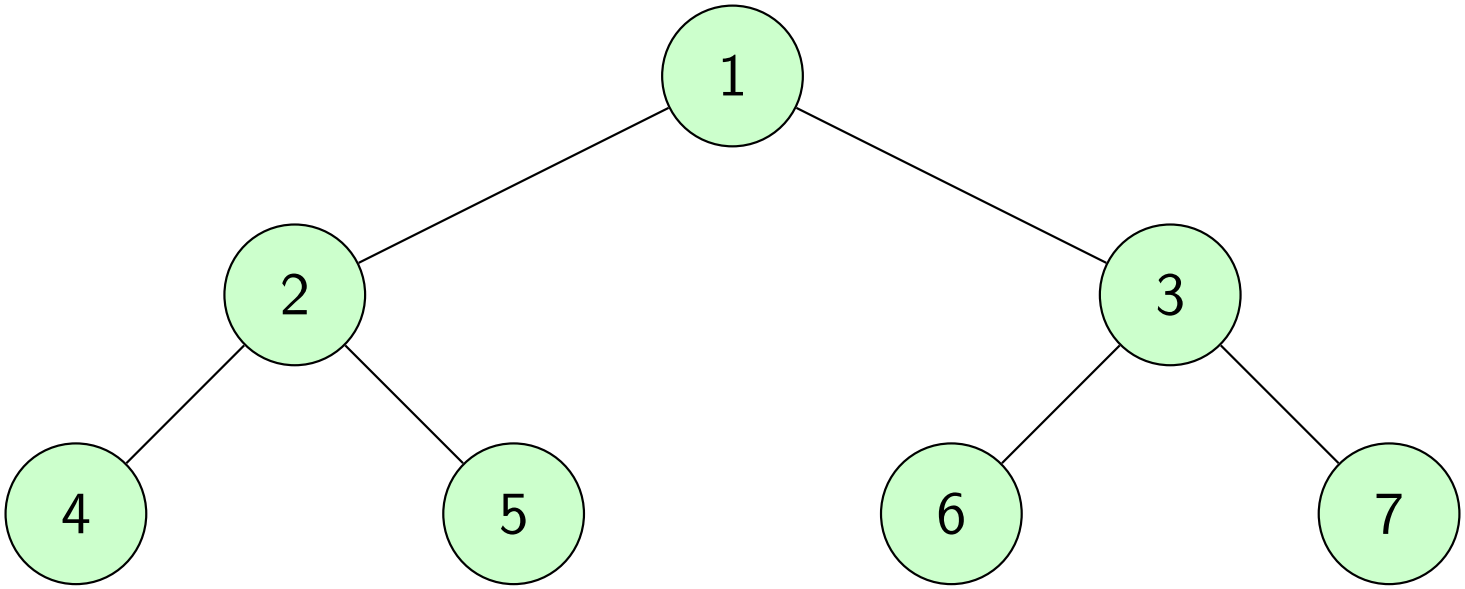
Cây là một đồ thị vô hướng, liên thông và **không có chu trình**.

Các tính chất tương đương

Nếu đồ thị G có n đỉnh, các điều sau là tương đương:

- G là một cây.
- G liên thông và có đúng $n - 1$ cạnh.
- Giữa 2 đỉnh bất kỳ có duy nhất một đường đi đơn.
- G không có chu trình nhưng thêm 1 cạnh bất kỳ sẽ tạo ra chu trình.

Ví dụ đồ thị dạng cây



Đặc điểm:

- Đồ thị liên thông
- Không chứa chu trình
- Với n đỉnh thì có đúng $n - 1$ cạnh

Ứng dụng thực tế

Kỹ thuật & Công nghệ

- **Routing:** Định tuyến gói tin trên Internet (Shortest Path).
- **Database:** Lập chỉ mục (Index) bằng cấu trúc cây.
- **AI:** Tìm kiếm trạng thái trong không gian bài toán (Game tree).

Đời sống & Khoa học

- **Logistics:** Tối ưu hóa quãng đường vận chuyển.
- **Sinh học:** Biểu diễn cấu trúc phân tử, ADN.
- **Quy hoạch:** Thiết kế mạng lưới điện, nước đô thị.

Cảm ơn thầy cô và các bạn đã theo dõi!

Xin chân thành cảm ơn

Tài liệu tham khảo

https://wiki.vnoi.info

Hà Minh Đức